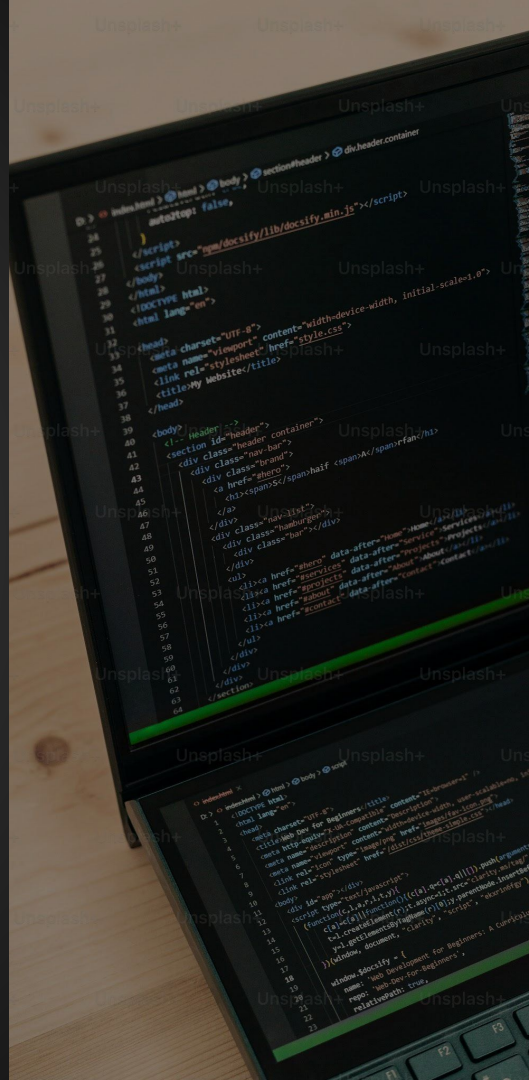
Desenvolvimento de Microserviços

Prof. Guilherme Alves

FACULDADE Donaduzzi

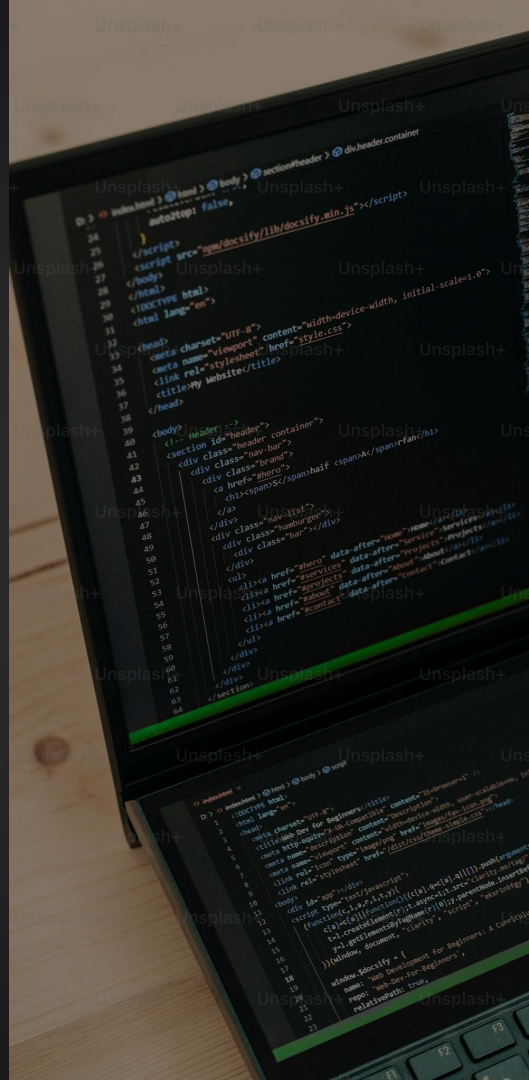
Tático
Estratégico



DDD Tático

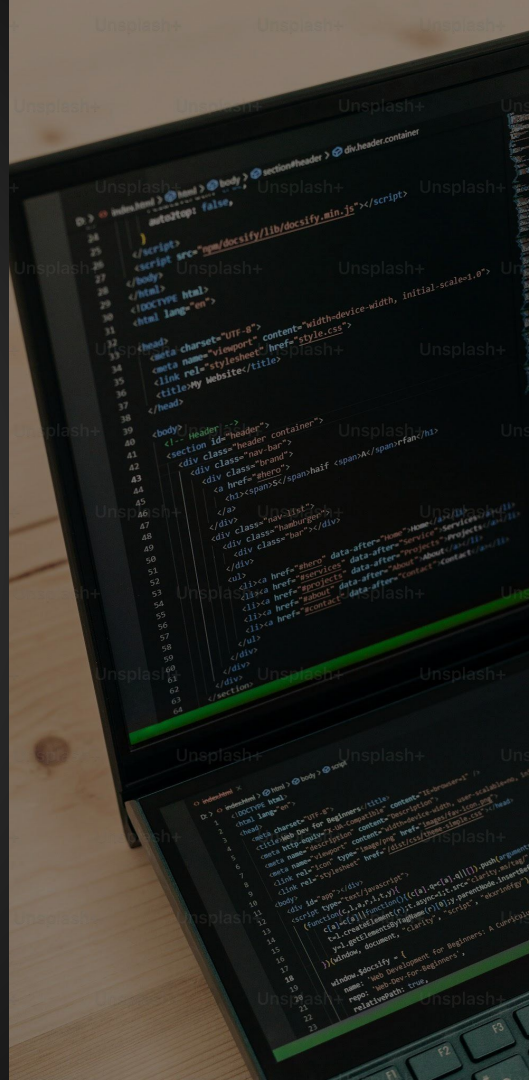
A camada tática é a parte que trata dos padrões de modelagem do domínio no nível de código, ou seja, como você implementa o modelo de negócio na prática.

É o conjunto de padrões e conceitos técnicos usados para modelar corretamente o domínio dentro de um **Bounded Context**.



Building Blocks

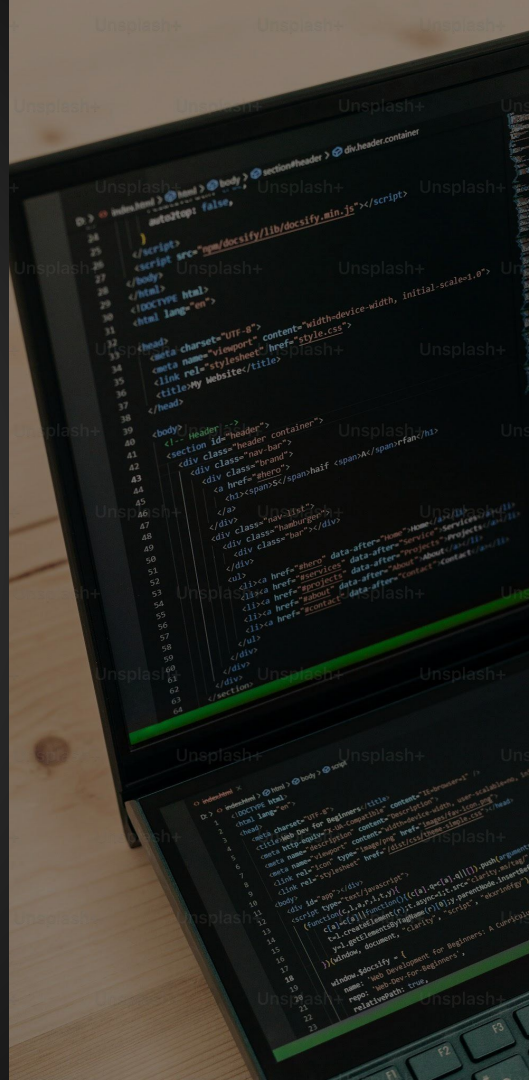
- **Entities** (*Entidades*)
- **Value Objects** (*Objetos de valor*)
- **Aggregates** (*Agregados*)
- **Repositories** (*Repositórios*)
- **Domain Services** (*Serviços de domínio*)
- **Factories** (*Fábricas*)



Entities

Uma entidade é um objeto do domínio que possui identidade própria e é distinguido dos demais não pelos seus atributos, mas pelo seu identificador.

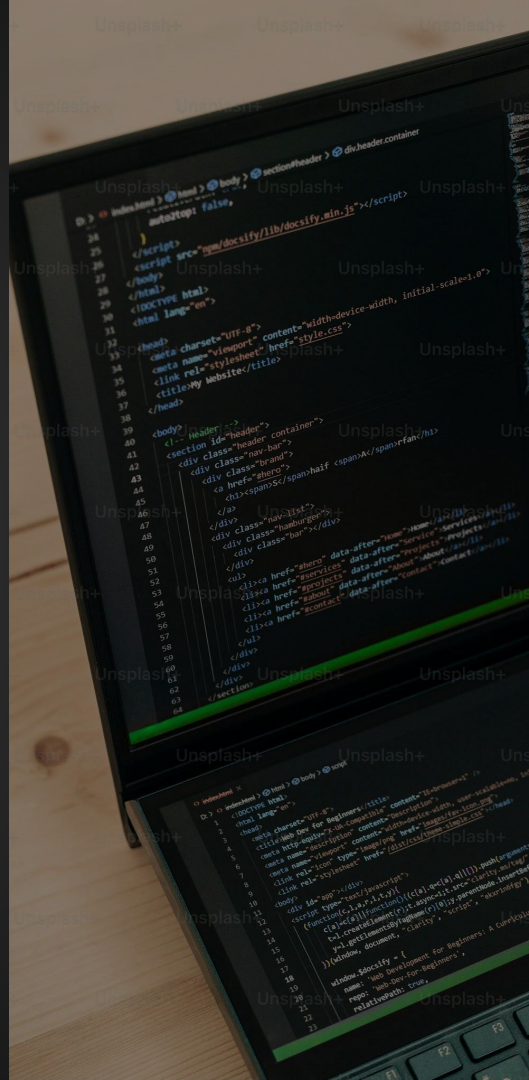
Resumidamente é uma classe que vai possuir um identificador (id) que diferencia as instâncias do objeto encapsulando suas responsabilidades (*objetos ricos escondem seus detalhes internos e tem responsabilidades sobre seus dados*).



Value Objects

Objeto que representa uma pequena entidade que descreve algum aspecto do domínio, sem identidade, sendo comparado apenas por seus atributos.

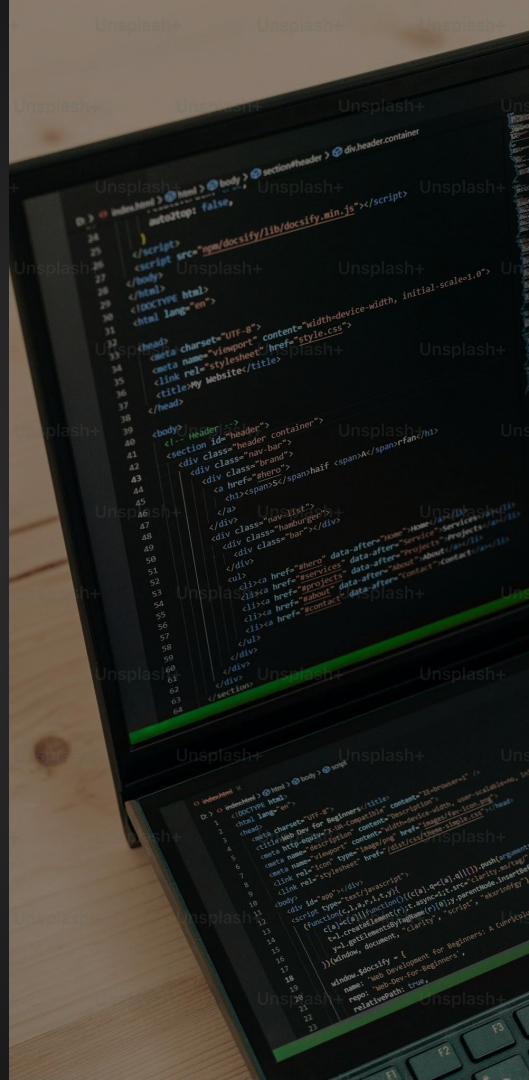
- Deixam o modelo mais expressivo
- Evitam validações espalhadas
- Agrupam conceitos



Aggregates

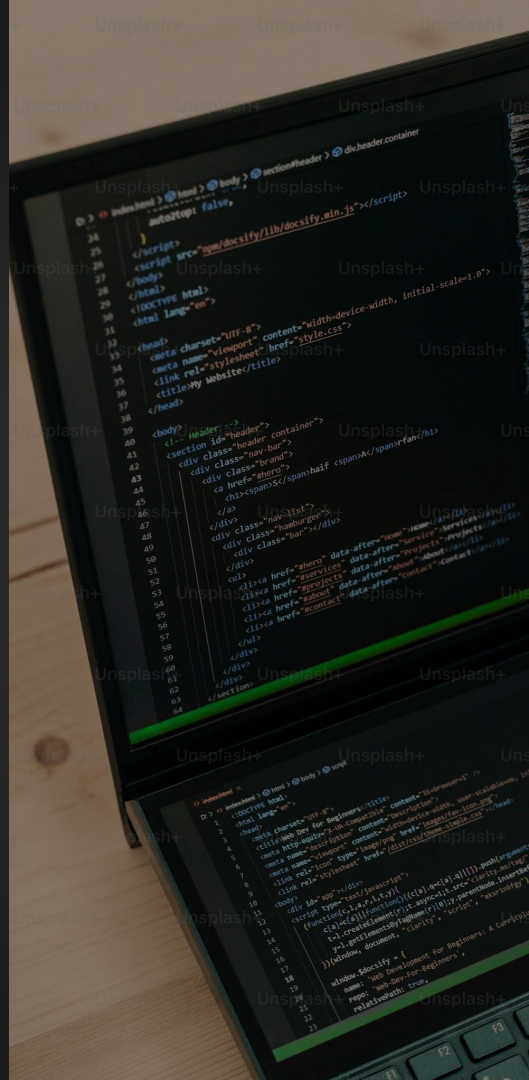
Um agregador é um conjunto de objetos do domínio (*entities + value objects*) que são tratados como uma única unidade de consistência.

- Garantir consistência das regras de negócio
- Definir limites de modificação
- Controlar invariantes
- Organizar o domínio em blocos coerentes
- Centraliza a entrada dos eventos



Aggregates

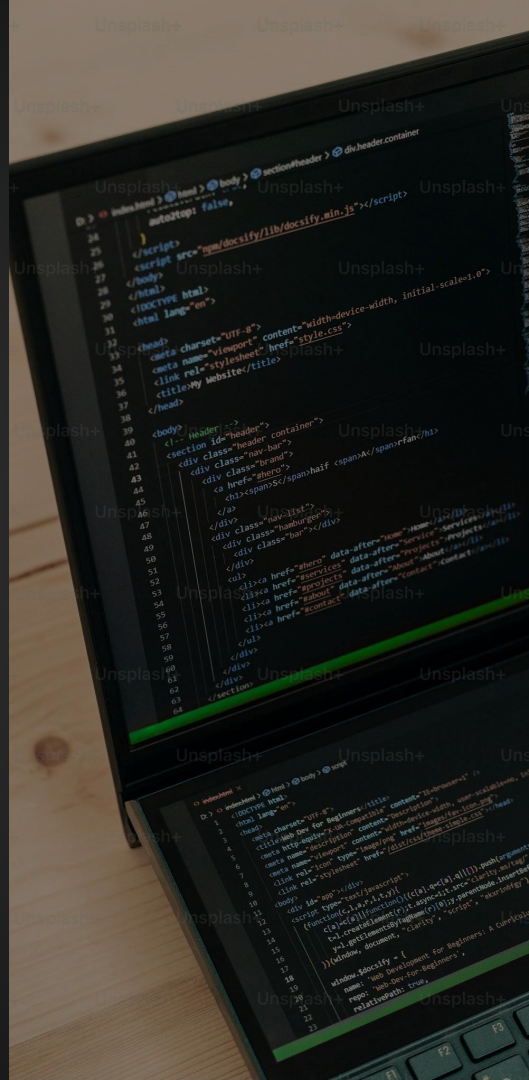
- Pedido (*Aggregate*)
  - Item Pedido (*Entity*)
  - Endereço Entrega (*Value Object*)
  - Status (*Value Object*)



Repositorios

Um repositório é um componente do domínio responsável por persistir e recuperar agregadores, escondendo os detalhes de infraestrutura.

- Funciona como uma coleção em memória conceitual de agregadores
- Persiste agregadores abstraindo o banco de dados
- Repository não é DAO genérico, ele trabalha com conceitos do domínio e não deve expor detalhes técnicos

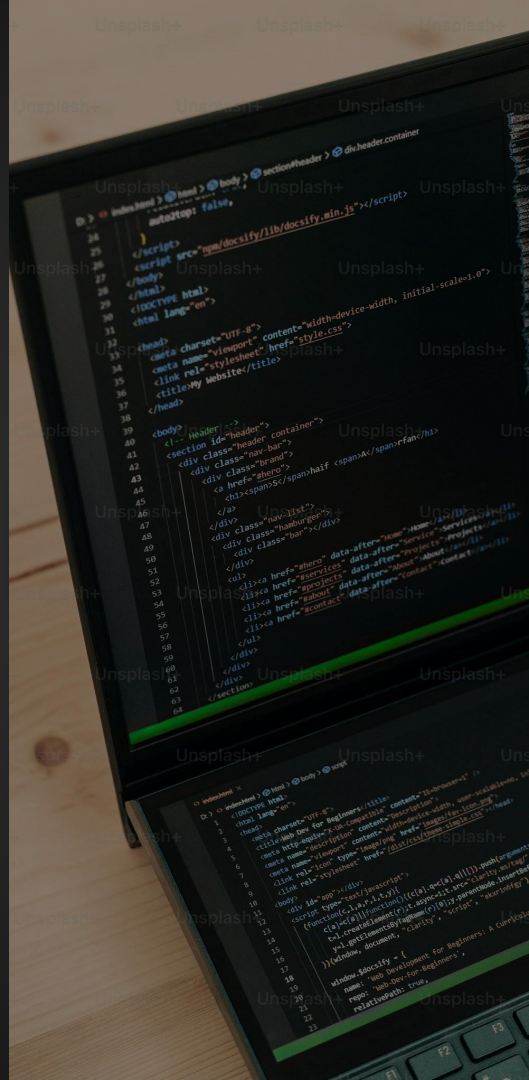


Domain Services

Um serviço do domínio é um componente do domínio usado para encapsular regras de negócio que não pertencem naturalmente a uma única entidade ou agregador.

Representa uma operação significativa do domínio que não pode ser atribuída naturalmente a um objeto específico.

- A regra envolve múltiplos agregadores
- A regra é conceitual, não estrutural
- Colocar a regra em uma entidade deixaria o modelo estranho
- A operação não pertence claramente a um objeto

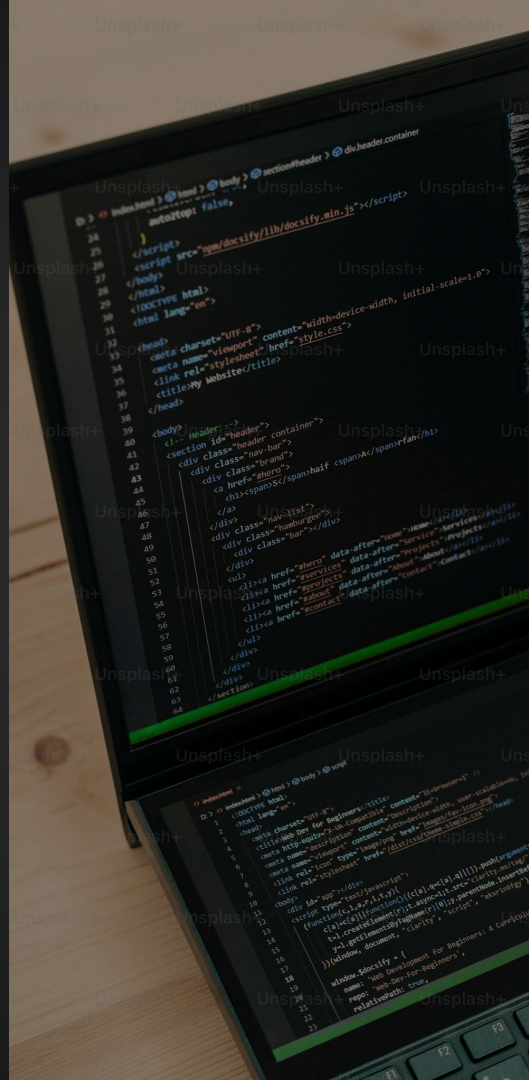


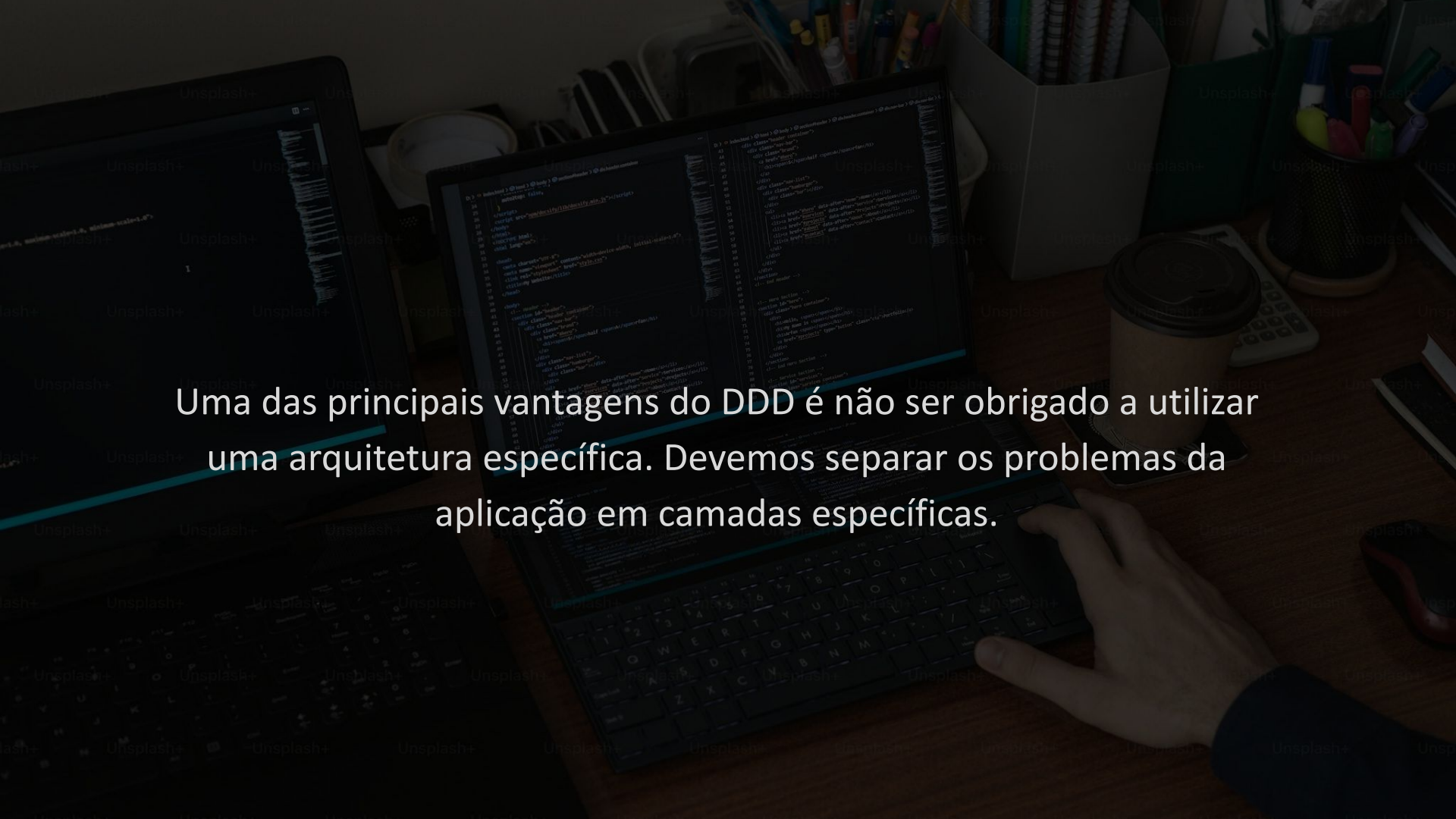
Factories

Uma factory é um padrão usado para criar entidades ou aggregates complexos, garantindo que todas as invariantes de negócio sejam respeitadas no momento da criação.

Uma factory encapsula a lógica de criação de objetos complexos do domínio, garantindo que eles já nasçam consistentes.

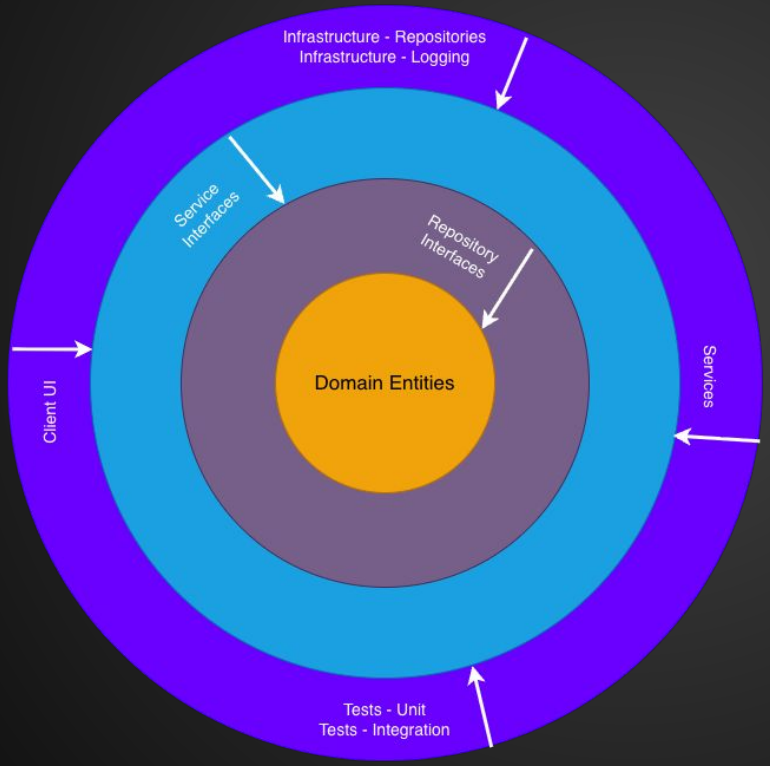
- Quando a criação envolve várias validações
- Quando a criação depende de múltiplos objetos
- Quando a criação envolve regras de negócio
- Quando o agregador precisa nascer consistente
- Quando o construtor ficaria poluído

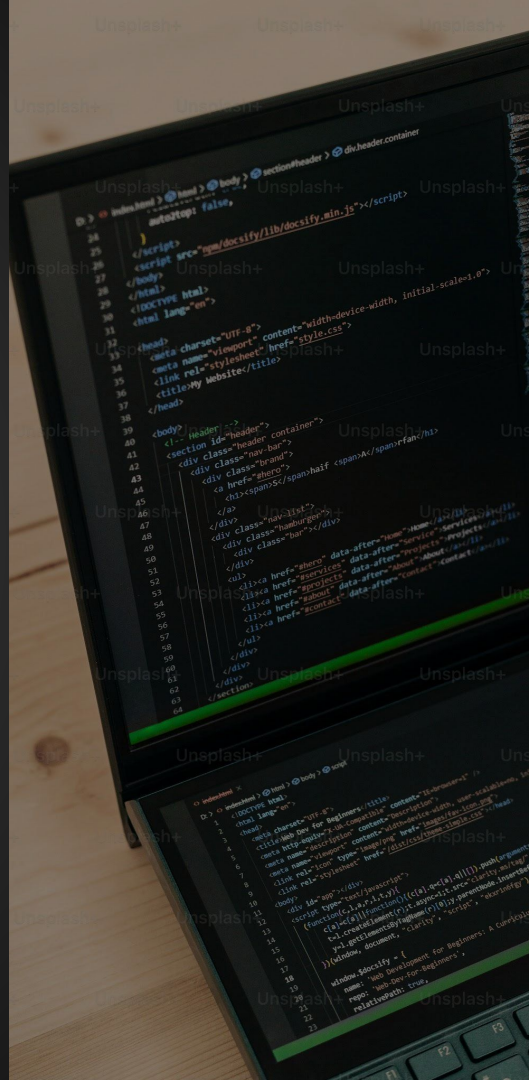


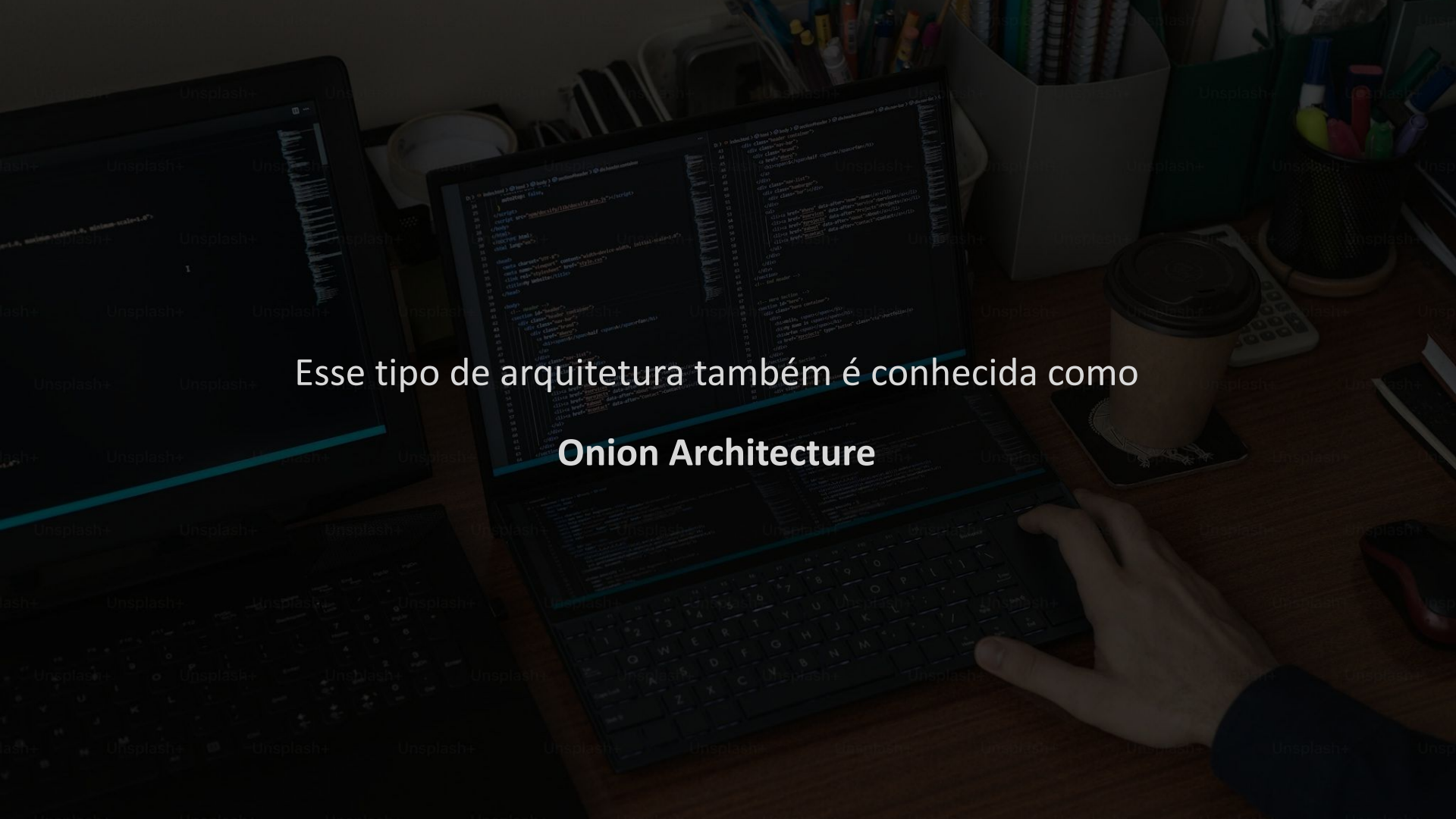


Uma das principais vantagens do DDD é não ser obrigado a utilizar uma arquitetura específica. Devemos separar os problemas da aplicação em camadas específicas.

Separação de camadas







Esse tipo de arquitetura também é conhecida como

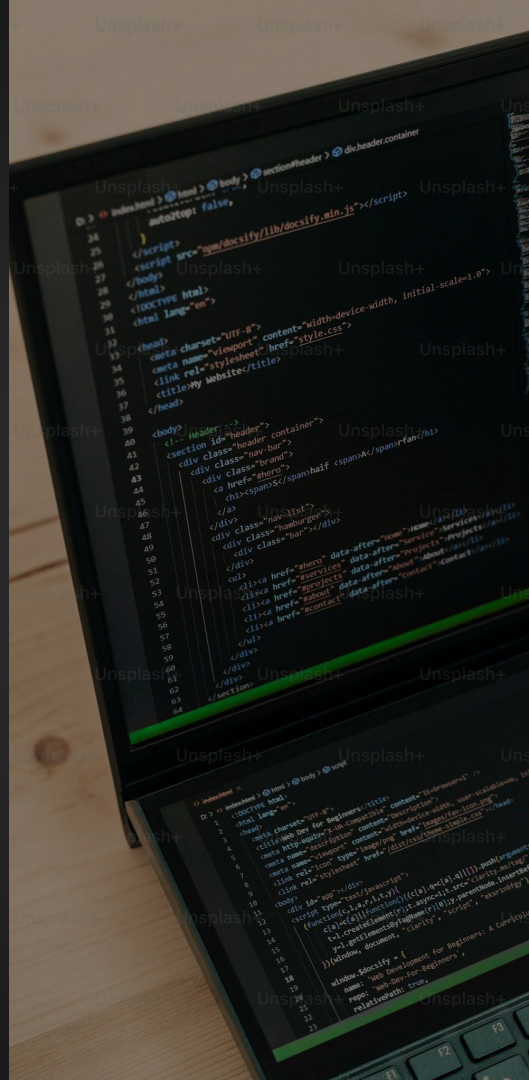
Onion Architecture

Onion Architecture

Descrita por Jeffrey Palermo em 2008

É baseada na aplicação das boas práticas do **OOP** (*Object-Oriented Programming*), assim como o uso do **SOLID**. O uso das técnicas de **IoC** (*Inversion of Control*) permitem quebrar a dependência cíclica e o alto acoplamento entre objetos e camadas.

Leia mais em: https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/



Quando falamos de system design o que buscamos independente da arquitetura é uma **alta coesão e baixo acoplamento**.

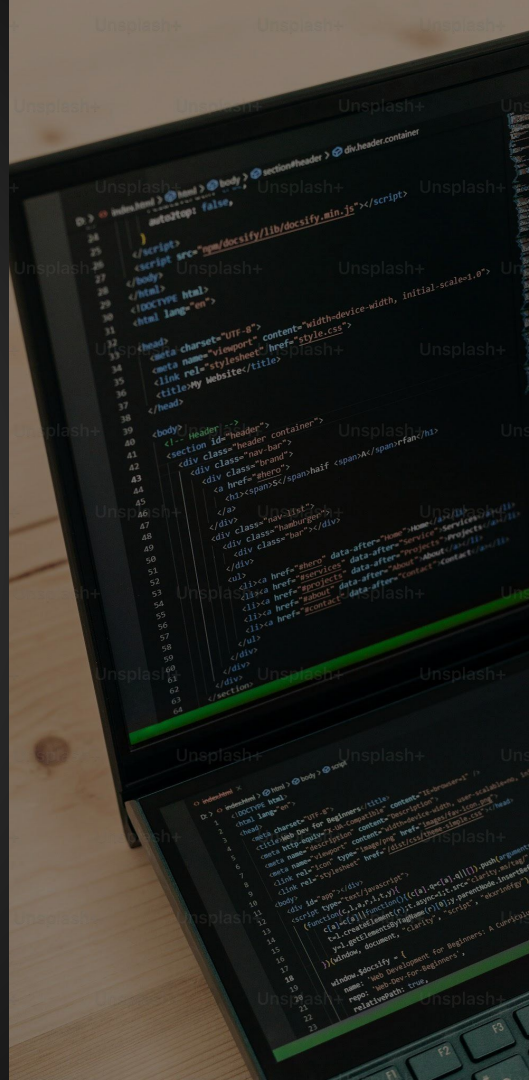
O acoplamento é o grau de dependência entre módulos e coesão é o grau de foco e responsabilidade única dentro de um módulo.

Arquiteturas

- **Hexagonal Architecture** por Alistair Cockburn em **2005**
- **Onion Architecture** por Jeffrey Palermo em **2008**
- **Clean Architecture** por Robert C. Martin em **2012**

Todos os conceitos são baseados na mesma idéia: Separação de responsabilidades, iniciando pelas camadas até chegar ao design de objetos.

A abstração desses conceitos é chamada de Clean Architecture e são esses conceitos que devem ser seguidos em qualquer arquitetura.



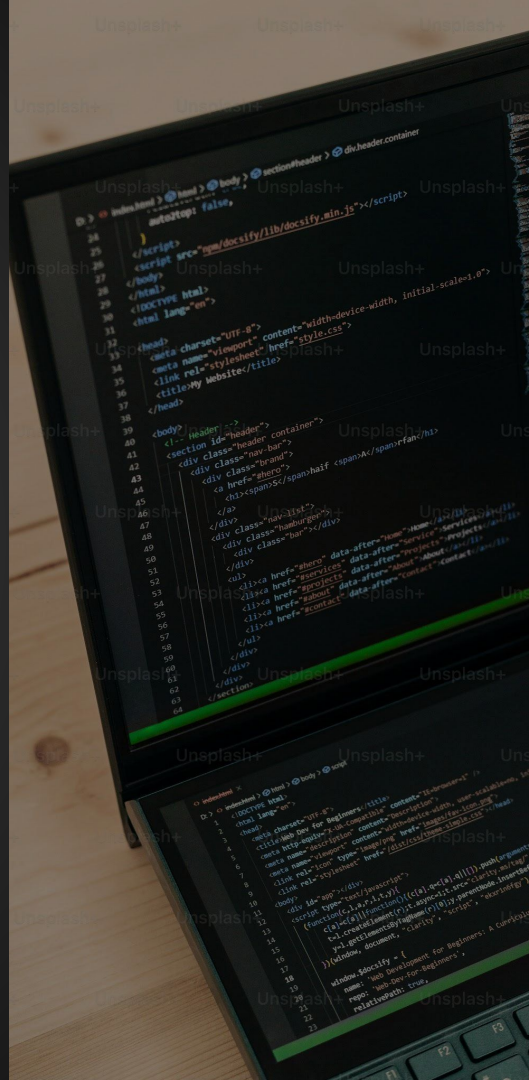
Hexagonal Architecture

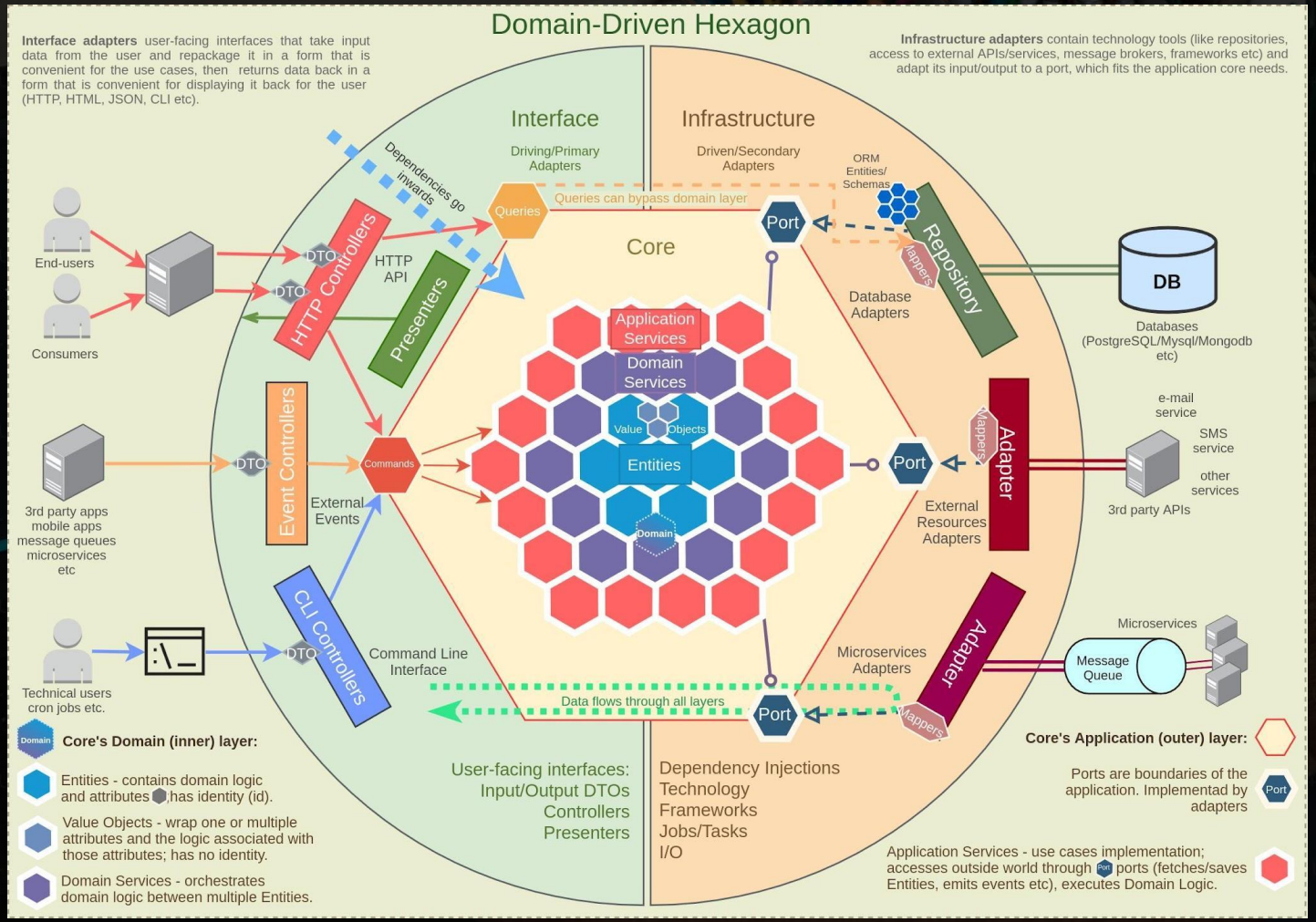
A arquitetura hexagonal separa a estrutura basicamente em três partes:

- **Core**
- **Ports** (*Interfaces*)
- **Adapters** (*Implementações externas*)

Os adaptadores convertem a linguagem externa ao domínio a uma linguagem interna.

Isso mantém a aplicação totalmente isolada, evitando que seja corrompida por dados externos mantendo o domínio livre de interferências.

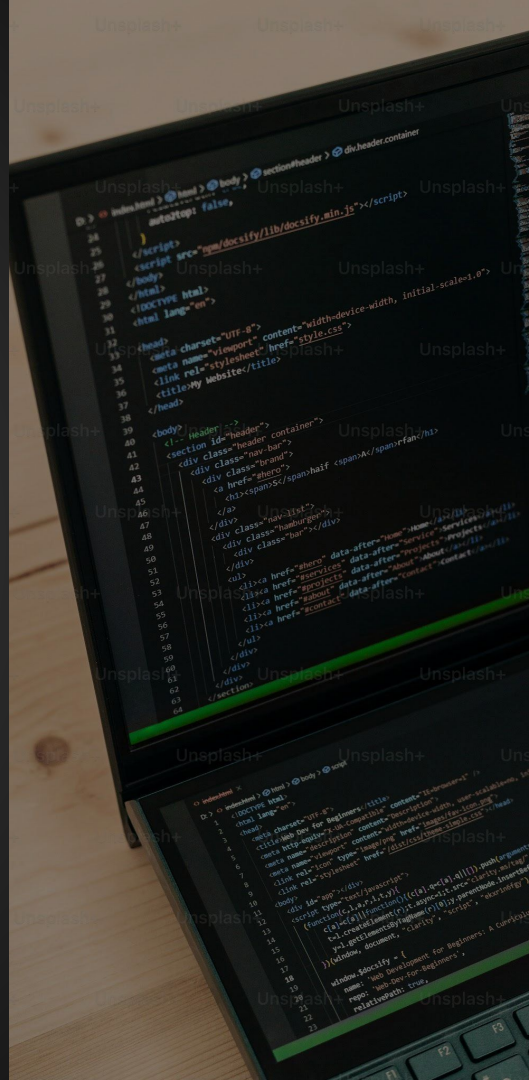


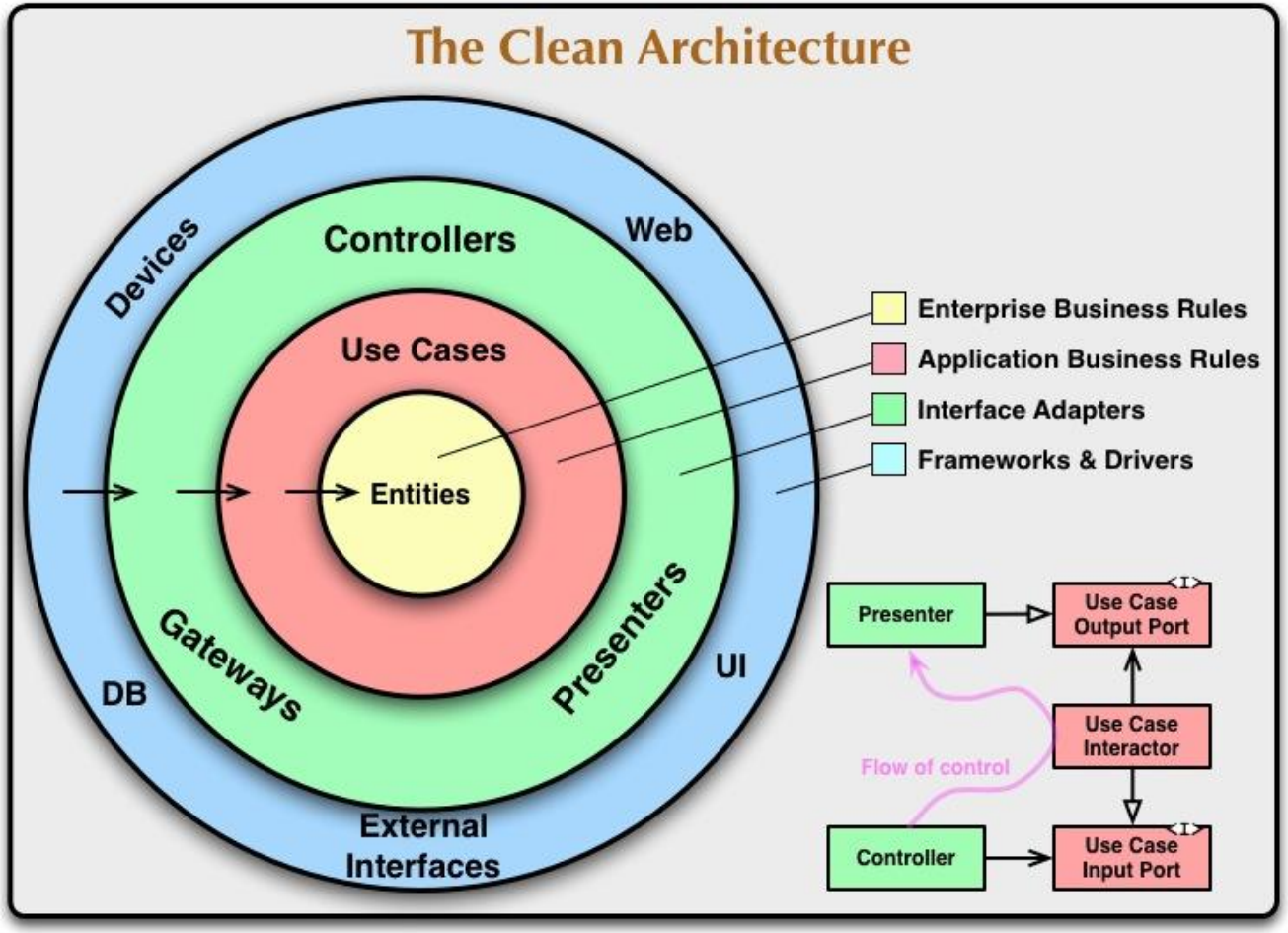

Domain-Driven Hexagon
Interface adapters user-facing interfaces that take input data from the user and repackage it in a form that is convenient for the use cases, then returns data back in a form that is convenient for displaying it back for the user (HTTP, HTML, JSON, CLI etc).
Infrastructure adapters contain technology tools (like repositories, access to external APIs/services, message brokers, frameworks etc) and adapt its input/output to a port, which fits the application core needs.
Interface
Infrastructure
Driving/Primary Adapters
Driven/Secondary Adapters
Dependencies go inwards
Queries
Queries can bypass domain layer
Core
End-users
Consumers
DTO
HTTP Controllers
HTTP API
Presenters
Port
ORM Entities/ Schemas
Repository
Mappers
Database Adapters
DB
Databases (PostgreSQL/Mysql/Mongodb etc)
Application Services
Domain Services
Value Objects
Entities
Domain
Commands
Event Controllers
External Events
3rd party apps mobile apps message queues microservices etc
Port
Adapter
External Resources Adapters
e-mail service
SMS service
other services
3rd party APIs
CLI Controllers
Command Line Interface
Technical users cron jobs etc.
Microservices Adapters
Adapter
Microservices
Message Queue
Data flows through all layers
Core's Domain (inner) layer:
Entities - contains domain logic and attributes, has identity (id).
Value Objects - wrap one or multiple attributes and the logic associated with those attributes; has no identity.
Domain Services - orchestrates domain logic between multiple Entities.
User-facing interfaces:
Input/Output DTOs
Controllers
Presenters
Dependency Injections
Technology
Frameworks
Jobs/Tasks
I/O
Core's Application (outer) layer:
Ports are boundaries of the application. Implementad by adapters
Application Services - use cases implementation; accesses outside world through ports (fetches/saves Entities, emits events etc), executes Domain Logic.

Clean Architecture

É baseada em algumas regras para criar um fluxo de dependências que possibilitam construir uma arquitetura com baixo acoplamento. Utiliza os princípios:

- SOLID
  - Single Responsibility Principle (**SRP**)
  - Open/closed Principle (**OCP**)
  - Liskov Substitution Principle (**LSP**)
  - Interface Segregation Principle (**ISP**)
  - Dependency Inversion Principle (**DIP**)
- Reuse/Release Equivalence Principle (**REP**)
- Common Closure Principle (**CCP**)
- Common Reuse Principle (**CRP**)
- Acyclic Dependency Principle (**ADP**)
- Stable Dependency Principle (**SDP**)
- Stable Abstraction Principle (**SAP**)



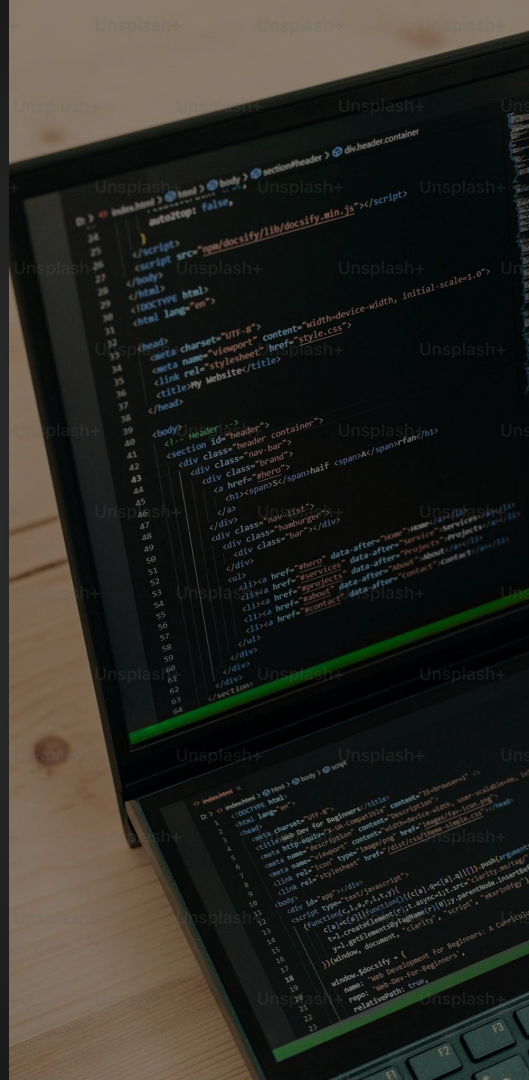

The Clean Architecture
Devices
Controllers
Web
Use Cases
Entities
Gateways
Presenters
DB
UI
External Interfaces
Enterprise Business Rules
Application Business Rules
Interface Adapters
Frameworks & Drivers
Presenter
Use Case Output Port
Use Case Interactor
Flow of control
Controller
Use Case Input Port

Resumindo...

Todos os conceitos são baseados na mesma idéia de separação de responsabilidades, iniciando pelas camadas até chegar ao design de objetos, isolando o domínio.

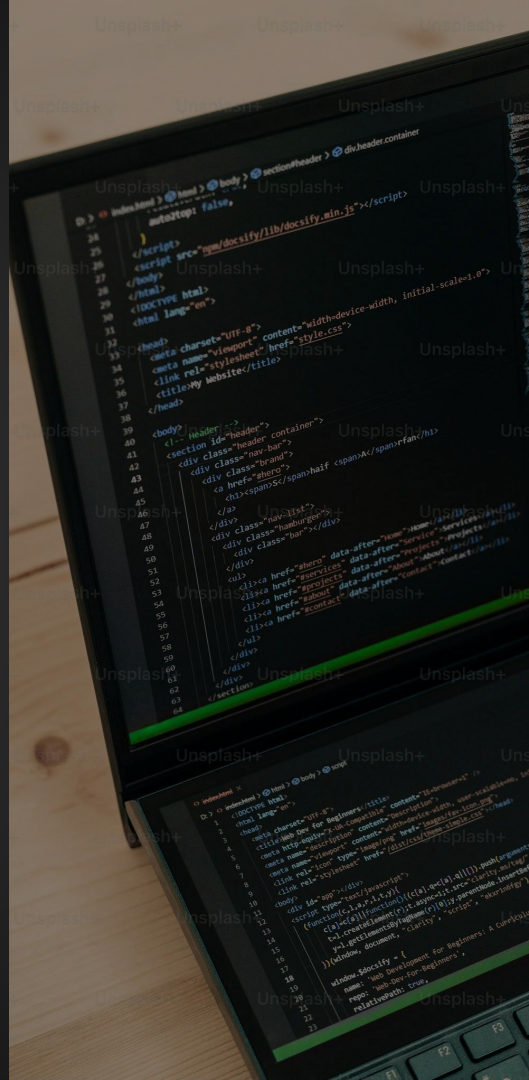
Todas as abordagens apresentadas podem ser implementadas da forma que for melhor para seu projeto.

Todas seguem o fluxo de dependências de fora para dentro.



Organizando o pensamento...

Não confunda domain do DDD com domain layer

- Cada microserviço deve ter um bounded context bem definido
- Dentro do microserviço, o domain layer representa o modelo do bounded context daquele serviço

De forma prática o bounded context define a fronteira do modelo, o domain layer implementa o modelo dentro dessa fronteira e o microserviço é a unidade de deploy que idealmente coincide com a fronteira.